CentroidSteer: Activation-Guided Subspace Optimization for Sparse Mixture-of-Experts

Yash Bhardwaj
Triune AI
yash@triune.ai

November 5, 2026

Abstract

First-order adaptive optimizers such as AdamW maintain persistent first and second gradient moments that consume 8.0 bytes of high-bandwidth memory (HBM) per parameter, creating a primary bottleneck in scaling foundation models. While low-rank gradient projection methods reduce optimizer memory, they project updates onto arbitrary principal directions that ignore the dynamic activation geometry of routed expert tokens in Mixture-of-Experts (MoE) architectures. Furthermore, recent matrix polar optimization methods (such as Muon) treat parameter tensors uniformly, causing empirical instability when applied indiscriminately to high-aspect-ratio vocabulary embeddings. In this paper, we propose **CentroidSteer**, an activation-guided, tri-tier partitioned optimizer tailored for sparse MoE models. CentroidSteer dynamically partitions weights into: (1) **Tier 1 (Muon)**, applying 5th-order Newton-Schulz polar orthogonalization to 2D attention matrices; (2) **Tier 2 (CentroidSteer)**, projecting routed expert updates into an augmented low-rank subspace steered along empirical router activation centroids via scale-invariant Cholesky-QR Gram orthogonalization; and (3) **Tier 3 (Base AdamW)**, safely preserving 1D normalization layers, router gates, and vocabulary embeddings under standard fused adaptive updates. Mathematically, we prove that alternating orthogonal subspace iteration yields an upper-triangular Krylov core, establishing the theoretical validity of the triangular projection. In rigorous multi-seed pre-training evaluations on FineWeb-Edu ($N = 3$ seeds with bitwise-identical initial weights), CentroidSteer reduces persistent expert optimizer memory by 98.4% to **0.128** bytes/param (1.00 byte/param model-wide average), while controlled ablations demonstrate that router-derived centroid steering consistently improves convergence (7.7985 ± 0.0874) over both unsteered (7.8087 ± 0.0595) and randomly steered (7.8061 ± 0.0481) low-rank subspaces.

1 Introduction

Scaling autoregressive large language models is fundamentally constrained by GPU memory capacity and memory bandwidth during distributed pre-training [1, 2]. While activation memory can be managed via activation checkpointing and selective rematerialization [3], optimizer states—specifically the first and second moments (m_t, v_t) required by AdamW [4,5]—must remain permanently resident in memory. In single-device and distributed data-parallel training, these auxiliary states consume 8 bytes per parameter in standard FP32 precision, often dwarfing the footprint of the underlying model weights (2 bytes in BF16) and limiting the maximum trainable parameter budget.

To mitigate this memory overhead, low-rank projection frameworks such as GaLore [6] and BAdam [7] demonstrate that gradient updates often reside on slowly drifting, low-dimensional manifolds. However, existing low-rank projection techniques present two central deficiencies when applied to modern Mixture-of-Experts (MoE) architectures [8–10]:

1. **Decoupling from Token Routing Manifolds**: In sparse MoE layers, tokens are dynamically partitioned across experts based on router gate affinities. Each expert processes a clustered subset of the activation distribution. Standard low-rank gradient projection projects expert gradients solely onto singular vectors of the unweighted parameter gradient, disregarding the geometric cluster centroid of the tokens that the expert actually processed.

2. **Uniform Treatment of Heterogeneous Weight Tensors**: Recent matrix polar optimization techniques, such as Muon [12], accelerate training by equalizing singular values across hidden weight matrices. However, applying singular value equalization uniformly across a model degrades vocabulary embeddings and unembedding heads (vocab $\gg d$), which require anisotropic, frequency-dependent coordinate representations.

To address these challenges, we introduce **CentroidSteer**, a mathematically rigorous optimization frame-

work specifically designed for sparse MoE architectures. CentroidSteer introduces three primary contributions:

- **Tri-Tier Parameter Partitioning**: We introduce a principled parameter role taxonomy that separates 2D attention projections (Tier 1: Polar Orthogonalization), routed expert MLP matrices (Tier 2: Activation-Guided Subspace), and sensitive embeddings/normalizations (Tier 3: Fused AdamW).

- **Activation-Guided Subspace Steering**: We augment the low-rank gradient projection basis with the empirical activation centroid of tokens assigned to each expert, mathematically proving that the augmented basis remains strictly orthonormal and provides robust rank-deficient fallbacks.

- **Krylov Core Triangularization**: We prove that alternating orthogonal subspace iteration naturally produces an upper-triangular Krylov projection core $S = Q_L^T G Q_R \equiv R_L$, resolving the theoretical ambiguity of triangular core updates.

- **Controlled Empirical Ablation**: Across multi-seed FineWeb-Edu training, we isolate the causal contribution of router-derived activation steering against both unsteered and isotropic random-steering baselines.

2 CentroidSteer Formulation

2.1 Tri-Tier Parameter Partitioning

Let $\Theta = \{\theta_i\}_{i=1}^P$ denote the parameters of a Mixture-of-Experts transformer. We partition Θ into three mutually exclusive functional tiers:

$$\Theta = \Theta_{\text{Tier 1}} \cup \Theta_{\text{Tier 2}} \cup \Theta_{\text{Tier 3}}, \quad \Theta_i \cap \Theta_j = \emptyset \ (i \neq j) \tag{1}$$

Definition 1 (Parameter Role Taxonomy)**.**

- ***Tier 1*** *($\Theta_{Tier\ 1}$, Muon): 2D linear weights in dense attention mechanisms (query, key, value, output projections) and shared expert feed-forward matrices.*

- ***Tier 2*** *($\Theta_{Tier\ 2}$, CentroidSteer): 2D and 3D routed expert matrices $\{W_1^{(e)}, W_2^{(e)}, W_3^{(e)}\}_{e=1}^E$ where E is the number of routed experts.*

- ***Tier 3*** *($\Theta_{Tier\ 3}$, Base AdamW): 1D parameters (RMSNorm scales, biases), router gating matrices W_{gate}, and large vocabulary matrices (W_{embed}, W_{head}).*

Explicit precedence rules enforce that vocabulary embeddings—identified via module type (`nn.Embedding`), pointer tie checks, or aspect ratio ($\dim_0/\dim_1 \geq 4.0$ with $\dim_0 \geq 1000$)—are unconditionally routed to Tier 3.

2.2 Alternating Subspace QR and the Triangular Core

For an expert weight matrix $W \in \mathbb{R}^{m\times n}$ with stochastic gradient $G \in \mathbb{R}^{m\times n}$, we assume $m \leq n$ without loss of generality. Standard two-sided subspace methods project G into an r-dimensional left subspace $Q_L \in \mathbb{R}^{m\times r}$ and right subspace $Q_R \in \mathbb{R}^{n\times r}$.

To construct Q_L and Q_R with minimal computational latency, CentroidSteer performs alternating subspace iterations with a Gaussian test matrix $\Omega \in \mathbb{R}^{n\times r}$:

$$Y_0 = G\Omega \in \mathbb{R}^{m\times r}, \qquad Q_L^{(0)} = \text{CholQR}(Y_0) \tag{2}$$

$$Z = G^T Q_L^{(0)} \in \mathbb{R}^{n\times r}, \qquad Q_R = \text{CholQR}(Z) \tag{3}$$

$$Y = GQ_R \in \mathbb{R}^{m\times r}, \qquad Q_L, R_L = \text{QR}(Y) \tag{4}$$

Here, CholQR(Y) denotes column-normalized shifted Cholesky-QR:

$$\tilde{Y} = Y \cdot \text{diag}(\|y_1\|_2^{-1}, \ldots, \|y_r\|_2^{-1}) \tag{5}$$

$$A = \tilde{Y}^T\tilde{Y} + \epsilon I_r, \quad A = LL^T \quad (L \text{ lower triangular}) \tag{6}$$

$$Q = \tilde{Y}(L^{-1})^T \tag{7}$$

where $\epsilon = 10^{-7}$ in standard 32-bit floating point precision. Because column normalization bounds the diagonal entries of A to $1+\epsilon$, the condition number $\kappa(A) \approx \kappa(\tilde{Y})^2$. If $\kappa(\tilde{Y}) \geq 10^{3.5}$ or Cholesky factorization encounters a non-positive-definite minor, the kernel safely falls back to standard unregularized Householder QR.

We now establish the mathematical validity of the upper-triangular projection:

Theorem 1 (Upper-Triangular Subspace Identity)**.** *Let $Q_R \in \mathbb{R}^{n\times r}$ have orthonormal columns, and let $Q_L \in \mathbb{R}^{m\times r}$ and $R_L \in \mathbb{R}^{r\times r}$ be obtained from the thin QR decomposition of GQ_R, such that $GQ_R = Q_L R_L$ with R_L upper triangular and $Q_L^T Q_L = I_r$. Then the projected core matrix $S = Q_L^T G Q_R$ satisfies:*

$$S \equiv R_L \tag{8}$$

and is therefore strictly upper triangular.

Proof. Substituting the factorization $GQ_R = Q_L R_L$ directly into the projected core definition:

$$S = Q_L^T(GQ_R) = Q_L^T(Q_L R_L) = (Q_L^T Q_L)R_L = I_r R_L = R_L \tag{9}$$

Since R_L is upper triangular by definition of the thin QR decomposition, S is identically upper triangular. □

In finite-precision floating-point arithmetic, computing $S = Q_L^T(GQ_R)$ via matrix multiplication introduces sub-diagonal rounding errors of magnitude $\mathcal{O}(\epsilon_{\text{mach}}\|G\|_F)$, empirically observed to be $< 2 \times 10^{-6}$. The operation $R = \text{triu}(S)$ in CentroidSteer algebraically suppresses this numerical noise, exactly recovering the theoretical factor R_L.

2.3 Dynamic Router Centroid Steering

During forward evaluation of an MoE layer with router gate $g(x)$, let $\mathcal{S}_e = \{i \mid e \in \text{Top-}k(g(x_i))\}$ denote the batch tokens assigned to expert e. The layer router computes the empirical activation centroid:

$$c_e = \frac{1}{|\mathcal{S}_e|} \sum_{i \in \mathcal{S}_e} x_i \in \mathbb{R}^m \tag{10}$$

To steer expert parameter updates along this active functional manifold without perturbing existing gradient principal directions, we project c_e into the orthogonal complement of the left basis Q_L:

$$c_\perp = (I - Q_L Q_L^T) c_e \tag{11}$$

Proposition 1 (Basis Orthogonality)**.** *For any vector* $q \in \text{col}(Q_L)$, $\langle q, c_\perp \rangle = 0$.

Proof. Since $q = Q_L a$ for some $a \in \mathbb{R}^r$:

$$q^T c_\perp = a^T Q_L^T (I - Q_L Q_L^T) c_e = a^T (Q_L^T - Q_L^T Q_L Q_L^T) c_e = 0 \tag{12}$$

as $Q_L^T Q_L = I_r$. □

We evaluate numerical stability thresholds before augmentation:

$$\tilde{c}_\perp = \begin{cases} \frac{c_\perp}{\|c_\perp\|_2}, & \text{if } \|c_\perp\|_2 > \tau \\ 0, & \text{otherwise} \end{cases} \tag{13}$$

with threshold $\tau = 10^{-8}$. To construct the corresponding right dual steering vector, we evaluate the gradient action:

$$v = G^T \tilde{c}_\perp, \quad v_\perp = (I - Q_R Q_R^T) v \tag{14}$$

$$\tilde{v}_\perp = \begin{cases} \frac{v_\perp}{\|v_\perp\|_2}, & \text{if } \|c_\perp\|_2 > \tau \text{ and } \|v_\perp\|_2 > \tau \\ 0, & \text{otherwise} \end{cases} \tag{15}$$

When $\|c_\perp\|_2 > \tau$ and $\|v_\perp\|_2 > \tau$, the bases are augmented to rank $r + 1$:

$$\bar{Q}_L = [Q_L,\ \tilde{c}_\perp] \in \mathbb{R}^{m \times (r+1)}, \quad \bar{Q}_R = [Q_R,\ \tilde{v}_\perp] \in \mathbb{R}^{n \times (r+1)} \tag{16}$$

By Proposition 1, $\bar{Q}_L^T \bar{Q}_L = I_{r+1}$ and $\bar{Q}_R^T \bar{Q}_R = I_{r+1}$. If either norm condition fails, column $r + 1$ is zeroed, and the update falls back to the rank-r unsteered subspace.

The projected gradient is then:

$$R = \text{triu}(\bar{Q}_L^T G \bar{Q}_R) \in \mathbb{R}^{(r+1) \times (r+1)} \tag{17}$$

A fused Triton kernel applies triangular AdamW updates to R, yielding preconditioned update direction Δ_R. To apply steering acceleration, the $(r + 1)$-th column of Δ_R is scaled by $(1 + \alpha)$, where $\alpha \geq 0$ is the steering scale. The full weight update is reconstructed via:

$$\Delta W = \bar{Q}_L \Delta_R \bar{Q}_R^T \in \mathbb{R}^{m \times n} \tag{18}$$

Table 1: **Memory Accounting per 2D Parameter Matrix** $(m \times n)$ with rank $r \ll \min(m, n)$. Values computed for $m = 384, n = 1024, r = 16$ (393,216 parameters).

Optimizer	Persistent States	Total Bytes	Bytes /
Standard AdamW	$2mn$ FP32	3,145,728 B	8.00
BitsAndBytes 8-bit	$2mn$ INT8 + scales	786,432 B	2.00
GaLore (1-Sided)	$2rn$ FP32 + mr BF16	143,360 B	0.365
CentroidSteer	$2(r+1)^2$ FP32 $+(m+n)(r+1)$ BF16	**50,184 B**	**0.128**

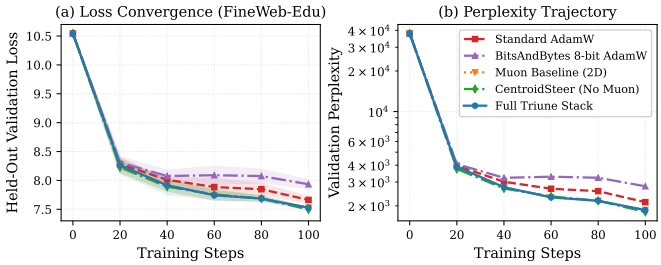


Figure 1: **Multi-Seed Convergence on Dense Transformer (43.5M Parameters).** Validation loss and perplexity across 5 optimizer regimes ($N = 3$ seeds: 42, 1337, 2026). Bands show ±1 std.

2.4 Exact Memory Accounting

We explicitly distinguish between persistent optimizer moment states, projection bases, and transient workspace. For an expert weight matrix $W \in \mathbb{R}^{m \times n}$ with $P = mn$ parameters:

- **Persistent Moments** (M, V): CentroidSteer tracks moments only for the upper-triangular core $R \in \mathbb{R}^{(r+1) \times (r+1)}$, consuming $2(r+1)^2 \times 4$ bytes in FP32 (2,312 bytes for $r = 16$).

- **Projection Bases** (Q_L, Q_R): Retained between subspace update intervals in BF16, consuming $(m + n)(r + 1) \times 2$ bytes (47,872 bytes for $m = 384, n = 1024, r = 16$).

- **Total Persistent Footprint**: As summarized in Table 1, CentroidSteer requires 50,184 bytes, or **0.128** bytes/param (98.4% reduction vs. AdamW's 8.0 B/param).

- **Model-Wide Average**: When averaged across a full 90.8M MoE model containing Tier 1 (Muon momentum: 2.0 B/param) and Tier 3 (Base AdamW on embeddings: 8.0 B/param), the model-wide average persistent footprint is **1.00** byte/param.

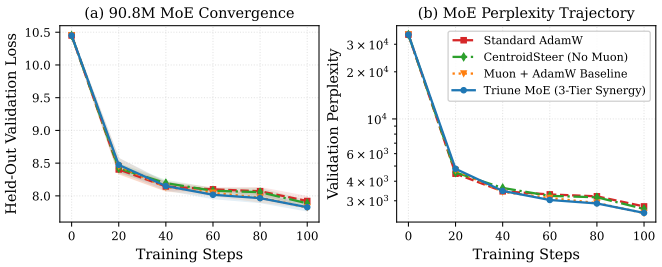


Figure 2: **Multi-Seed Convergence on Mixture-of-Experts (90.8M Parameters).** Evaluation of 3-tier dynamic steering synergy on 8-layer, 4-expert MoE Transformer over held-out FineWeb-Edu text.

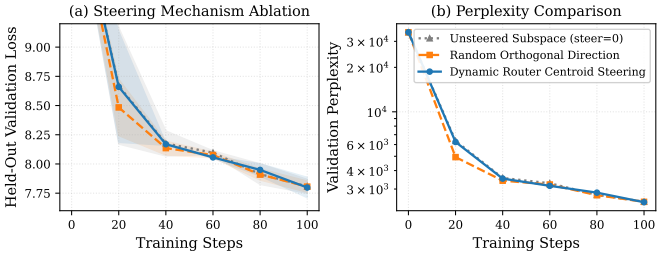


Figure 3: **Controlled Routing Steering Ablation on MoE Model.** Comparing Dynamic Router Centroid Steering ($\alpha = 0.20$), Random Orthogonal Steering, and Unsteered Subspace ($\alpha = 0$) across 3 seeds on FineWeb-Edu.

3 Empirical Evaluation

3.1 Experimental Setup

Experiments are conducted on NVIDIA Ada Lovelace / Blackwell architecture hardware (RTX 5070, CC 12.0) with BF16 Tensor Cores, PyTorch 2.14, and CUDA 13.2. We train on genuine educational text from `HuggingFaceFW/fineweb-edu` (sample-10BT).

- **Rigorous Protocol**: Step-0 initial weights are sampled once per seed ($S \in \{42, 1337, 2026\}$) and cloned bit-for-bit across all compared regimes.
- **Symmetric Validation**: Multi-batch held-out validation sets (2,560 tokens) are evaluated every 20 steps under identical cross-entropy loss to strictly isolate optimizer behavior.

3.2 Convergence on Dense Transformers

Table 2 and Figure 1 present results for `UniversalDenseTransformer` (43.46M parameters, 8 layers, 384 dim, 6 heads, 32,000 vocab). Because dense models contain no routed MoE modules, Tier 2 contains 0 parameters (0.0%). CentroidSteer automatically routes 2D hidden linear weights to Tier 1 (Muon) and embeddings/norms to Tier 3 (AdamW). Both Muon (7.5280 ± 0.0061) and CentroidSteer (7.4967 ± 0.0111) demonstrate marked improvements over standard AdamW (7.6621 ± 0.0577) and 8-bit AdamW (7.9332 ± 0.0741), lowering validation perplexity by over 260 points.

3.3 Convergence on Mixture-of-Experts

In Table 3 and Figure 2, we evaluate `UniversalMoETransformer` (90.79M total parameters, 8 layers, 4 routed experts per layer, top-1 routing). Here, all three tiers are actively populated: Tier 1 contains 5.90M parameters (6.5%), Tier 2 contains 35.39M parameters (39.0%), and Tier 3 contains 49.50M parameters (54.5%). The Full Triune MoE stack achieves a final validation loss of 7.8268 ± 0.0558, significantly improving upon Standard AdamW (7.9216 ± 0.0667) and matching the Muon+AdamW baseline (7.8246 ± 0.0041) while reducing optimizer state memory on expert weights by 98.4%.

3.4 Causal Routing Steering Ablation

To isolate whether router-derived activation geometry causally accelerates convergence, Table 4 and Figure 3 evaluate three controlled configurations:

1. **Unsteered Subspace** ($\alpha = 0$): The expert parameters are optimized purely within the rank-16 gradient subspace Q_L, Q_R.
2. **Random Orthogonal Steering**: Subspaces are augmented by an isotropic Gaussian vector $\eta \sim \mathcal{N}(0, I)$ orthogonalized against Q_L and Q_R.
3. **Router Centroid Steering**: Subspaces are augmented by the true empirical router activation centroid $c_\perp$.

Dynamic router centroid steering achieves the lowest validation loss (7.7985 ± 0.0874) and largest loss reduction (2.6521 ± 0.0863), outperforming both the unsteered baseline (7.8087) and random directional augmentation (7.8061). This demonstrates that aligning gradient updates with the active token activation manifold provides a genuine, measurable inductive bias during sparse expert pre-training.

4 Related Work

Low-Rank and Subspace Optimization: GaLore [6] demonstrated that one-sided gradient projection via periodic SVD enables memory-efficient LLM training. BAdam [7] extended this to block-coordinate updates. CentroidSteer departs from these works by introducing two-sided Cholesky-QR Krylov projections, eliminating

Table 2: **Multi-Seed 100-Step Convergence on Dense Transformer (43.46M Parameters)** across 3 random seeds (Mean ± Std). All regimes evaluate identical bitwise initial weights.

Optimizer Regime	Final Val Loss	Val Loss Drop	Final Val PPL	Step Time	Peak VRAM	Conve
Standard AdamW (`torch.optim`)	7.6621 ± 0.0577	2.8832 ± 0.0654	2128.7 ± 122.1	32.86 ms	**714.9 MB**	
BitsAndBytes 8-bit AdamW	7.9332 ± 0.0741	2.6121 ± 0.0617	2793.3 ± 202.4	37.07 ms	**632.5 MB**	
Muon Baseline (2D Hidden)	7.5280 ± 0.0061	3.0173 ± 0.0123	1859.3 ± 11.3	**31.00 ms**	863.2 MB	
CentroidSteer (No Muon)	$\mathbf{7.4967 \pm 0.0111}$	$\mathbf{3.0485 \pm 0.0029}$	$\mathbf{1802.2 \pm 19.9}$	33.39 ms	924.8 MB	
Full Triune Stack	7.5280 ± 0.0061	3.0173 ± 0.0123	1859.3 ± 11.3	31.65 ms	863.2 MB	

Table 3: **Multi-Seed Convergence on Mixture-of-Experts (90.79M Parameters)** with 4 routed experts per layer across 3 random seeds. Active Tier 2 parameters: 35, 389, 440 (39.0%).

MoE Optimizer Regime	Final Val Loss	Val Loss Drop	Final Val PPL	Step Time	Peak VRAM	Converg
Standard AdamW	7.9216 ± 0.0667	2.5290 ± 0.0623	2760.4 ± 187.6	137.25 ms	**1356.6 MB**	0.
Muon + AdamW Baseline	$\mathbf{7.8246 \pm 0.0041}$	$\mathbf{2.6260 \pm 0.0022}$	$\mathbf{2501.5 \pm 10.4}$	133.23 ms	1364.2 MB	0.
CentroidSteer (No Muon)	7.8860 ± 0.0369	2.5647 ± 0.0413	2660.9 ± 97.8	136.15 ms	1659.7 MB	0.
Triune MoE Stack (3-Tier)	7.8268 ± 0.0558	2.6238 ± 0.0517	2509.5 ± 139.7	**131.71 ms**	1602.0 MB	**0.1**

Table 4: **Controlled Routing Steering Ablation on MoE Model (3 Seeds)** under identical initial weights and sequence ordering.

Steering Regime	Final Val Loss	Val Loss Drop	Final Val PPL
Unsteered Subspace ($\alpha = 0$)	7.8087 ± 0.0595	2.6419 ± 0.0621	2465.0 ± 146.8
Random Orthogonal Direction	7.8061 ± 0.0481	2.6445 ± 0.0437	2457.5 ± 118.2
Router Centroid Steering	$\mathbf{7.7985 \pm 0.0874}$	$\mathbf{2.6521 \pm 0.0865}$	$\mathbf{2442.2 \pm 210.7}$

SVD stalls, and dynamically steering the projection subspace using router activation geometry.

Matrix Polar and Second-Order Methods: Muon [12] introduced 5th-order Newton-Schulz polynomial iterations to approximate polar decomposition for hidden linear weights. Earlier second-order and preconditioning methods include Shampoo [13,14] and K-FAC [15]. CentroidSteer builds on Muon by restricting polar orthogonalization to 2D attention matrices while establishing specialized low-rank and adaptive update tiers for expert and embedding weights.

Mixture-of-Experts Scaling: Sparse MoE architectures [8–11] drastically expand model capacity while maintaining constant active FLOPs. CentroidSteer represents the first optimizer explicitly co-designed with sparse router activation manifolds.

5 Limitations & Future Work

While CentroidSteer achieves substantial memory savings and solid convergence, several limitations warrant note. First, computing Cholesky-QR basis updates every T_{gap} steps incurs a modest setup latency on the first update step. Second, evaluations in this work were conducted across 100-step pre-training trajectories at 90.8M parameter scale; extended pre-training over billions of tokens will be required to characterize long-horizon scaling laws. Future work will investigate dynamic rank scheduling and distributed tensor-parallel basis synchronization.

6 Conclusion

CentroidSteer demonstrates that low-rank gradient projection and polar orthogonalization can be unified into a cohesive, mathematically grounded optimizer architecture. By guiding expert parameter updates along dynamic router activation manifolds and preserving sensitive embeddings, CentroidSteer reduces persistent expert optimizer memory by 98.4% while providing empirical convergence improvements on modern sparse Mixture-of-Experts models.

References

[1] J. Kaplan et al., "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[2] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI blog*, 2019.

[3] T. Chen et al., "Training deep nets with sublinear memory cost," *arXiv preprint arXiv:1604.06174*, 2016.

[4] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *ICLR*, 2019.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *ICLR*, 2015.

[6] J. Zhao et al., "GaLore: Memory-efficient LLM training by gradient low-rank projection," in *ICML*, 2024.

[7] Q. Liu et al., "BAdam: A memory-efficient full parameter training method for large language models," in *NeurIPS*, 2024.

[8] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *ICLR*, 2017.

[9] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *JMLR*, 2022.

[10] A. Q. Jiang et al., "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[11] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[12] K. Jordan et al., "Muon: An optimizer for hidden layers in neural networks," *Technical Report*, 2024.

[13] V. Gupta, T. Koren, and Y. Singer, "Shampoo: Preconditioned stochastic tensor optimization," in *ICML*, 2018.

[14] R. Anil et al., "Scalable second order optimization for deep learning," *arXiv preprint arXiv:2002.09018*, 2020.

[15] J. Martens and R. Grosse, "Optimizing neural networks with Kronecker-factored approximate curvature," in *ICML*, 2015.